FreeRADIUS bazlı NAC (Network Access Control) Sistemi

Teknolojiler:	Docker, Python 3.13 (FastAPI), FreeRADIUS 3.2, PostgreSQL 18, Redis 8
Konu:	Güvenlik ve Ağ İletişimi
Tarih:	Mart 2026
Ad Soyad:	Eren Şimşir

1. Giris

Bu staj ödevi, RADIUS protokolünü kullanarak temel düzeyde çalışan bir Network Access Control (NAC) sistemi kurmak amaciyla gelistirilmistir. Sistemin temel görevi, bir ağa bağlanmak isteyen kullanıcıları ve cihazları tanımlamak, yetkilendirmek ve oturum bilgilerini kayıt altına almaktır. Bunlar AAA (Authentication, Authorization, Accounting) olarak adlandırılır.

Projenin kapsamı: Docker Compose ile yönetilen dört servisli bir altyapıda, FreeRADIUS sunucusuyla kimlik doğrulamanın yapılması; FastAPI tabanlı bir policy engine üzerinden de yetkilendirme ve hesap yönetimi süreçlerinin yürütülmesidir. Sistem, hem şifreye dayalı (PAP) hem de MAC adresine dayalı (MAB) doğrulama yöntemlerini destekliyor.

2. Mimari Tasarim

2.1 Genel Mimari

Sistem dört temel servisten oluşuyor. Her biri bir Docker container olarak çalışır ve aralarındaki iletişim, 172. ile başlayan özel bir Docker ağı üzerinden kurulur."20.0.""0/16) üzerinden sağlanır :

Servis	Teknoloji	Gorev
freeradius	FreeRADIUS 3.2	RADIUS sunucusu — auth/authz/acct portlari (1812/1813 UDP)
api	Python 3.13 + FastAPI	Policy engine — REST API ile karar verme
postgres	PostgreSQL 18	Kalici veri: kullanicilar, gruplar, accounting kayitlari
redis	Redis 8	Gecici veri: aktif oturumlar, rate-limit sayaclari

2.2 Veri Akisi

Bir kullanıcı ağa bağlanmak istediğinde süreç genelde şöyle ilerler: (1) NAS cihazı (switch, AP vb.")"
FreeRADIUS'a bir Access-Request paketi gönderir. "(2) FreeRADIUS, rlm_rest modülüyle FastAPI'deki /authorize endpoint'ine bir istek gönderir ve kullanıcının grubuna bakarak VLAN bilgilerini
alır."

"(3) Sonra /auth endpoint'ine bir istek gönderip şifreyi doğrular." "API 200 dönünce Accept olur mu,"
401 hatası yüzünden istek reddediliyor gibi görünüyor. "(4) Doğrulama başarılı olduktan sonra NAS cihazı muhasebe paketlerini (Start, Interim-Update, Stop) gönderir. Bu paketler /accounting endpoint'i üzerinden PostgreSQL'e kaydedilir ve aktif oturumlar da Redis'te önbelleğe alınır.

2.3 Docker Compose Yapilandirmasi

Tüm servisler docker-compose dosyasında yapılandırdım genel olarak. Containerlar birbirleriyle iletişim halindedir.

2.4 FreeRADIUS Yapilandirmasi

Projeyi kurarken FreeRADIUS'un hantal varsayılan konfigürasyonunu bir kenara bırakıp, tamamen kendi hazırladığım `sites-enabled/default` dosyası üzerinden ilerledim. Sistem şöyle çalışıyor: **Authorize** kısmında önce `rest` modülünü çağırarak API'den kullanıcının VLAN ve politika bilgilerini çekiyorum, hemen ardından `Auth-Type` değerini `rest` olarak set ediyorum. **Authenticate** aşamasında ise sistem direkt `/auth` endpoint'ine gidiyor. Burada kararı `rlm_rest` modülü veriyor; eğer API'den **2xx** kodu dönerse erişim onaylanıyor (Accept), **4xx** gelirse reddediliyor (Reject). Ayrıca JSON tarafında yaşanabilecek karakter veya encoding sıkıntılarıyla uğraşmamak için `body = 'post'` ayarını kullandım; böylece veriler API'ye tertemiz bir şekilde **form-encoded** olarak gidiyor.

Bir de kurulum aşamasında (Dockerfile'da), bu projede işimize yaramayacak olan `inner-tunnel` ve `eap` modüllerini tamamen ayıkladım. Bunu yapmamın asıl sebebi, hem olası port çakışmalarının önüne geçmek hem de sistemi ayağa kaldırırken karşımıza çıkabilecek gereksiz EAP hatalarını en baştan engellemekti.

2.5 FastAPI Policy Engine

API, FreeRADIUS'un rlm_rest modülü kullanır. Temel endpoint'ler:

Endpoint	Metot	Islev
/auth	POST	PAP sifre dogrulama + MAB (MAC dogrulama)
/authorize	POST	Kullanici grubuna gore VLAN ve politika attribute'leri donme
/accounting	POST	Oturum verisi kaydetme (Start/Interim-Update/Stop)
/users	GET	Kullanici listesi ve online durumu
/sessions/active	GET	Redis'teki aktif oturumlari sorgulama
/devices	GET/POST	MAC cihaz yonetimi
/dashboard	GET	Monitoring dashboard (HTML)
/dashboard/data	GET	Dashboard icin JSON ozet veriler

API tarafında işleri kolaylaştırmak için yazdığım `parse_body()` fonksiyonu aslında projenin joker elemanı. Çünkü **FreeRADIUS (rlm_rest)** verileri bize **form-encoded** formatında gönderirken, sistemi dışarıdan **curl** veya benzeri araçlarla test ederken genelde **JSON** kullanıyoruz. Bu fonksiyon sayesinde her iki formatı da sorunsuz kabul edebiliyoruz.

Özellikle `/auth` endpoint'inde dikkat ettiğim bir detay var: Doğrulama başarılı olduğunda dışarıya sadece boş bir JSON objesi `{}` dönüyorum. Bunun sebebi tamamen **rlm_rest** modülünün çalışma mantığı. Eğer yanıtın içine herhangi bir **attribute** (nitelik) koysaydım, modül bunu 'updated' (güncellendi) olarak algılayıp

süreci karıştırabiliyordu. Boş bir body döndüğümde ise doğrudan 'ok' cevabını veriyor, bu da FreeRADIUS tarafında **Access-Accept** (erişim onayı) olarak karşılık buluyor.
Bu ufak ama kritik dokunuşla, sistemin gereksiz yere reddedilmesini veya karmaşık mantık hatalarına düşmesini engellemiş oldum.

2.6 Veritabani Semasi

Veritabanı tarafında sistemi **PostgreSQL** üzerinde toplam 7 ana tabloyla kurguladım. Yapıyı kurarken sadece veriyi saklamayı değil, FreeRADIUS'un çalışma mantığına tam uyumlu bir şema oluşturmayı hedefledim:

- **Kullanıcı ve Yetki Yönetimi:** `radcheck` tablosunda kimlik bilgilerini tutarken, `radreply` ile kullanıcıya özel dönülecek yanıtları ayırdım. Kullanıcıları mantıksal kümelere bölmek için `radusergroup` ve bu gruplara toplu VLAN/politika atamak için `radgroupreply` tablolarını kullandım.
- **Cihaz ve Erişim Kontrolü:** MAB süreçleri için `mac_devices` tablosunu ekledim; ağdaki switch veya AP gibi uç noktaları ise `nas` tablosunda tanımladım.
- **Kayıt ve Performans:** Tüm oturum geçmişini `radacct` tablosuna saniye bazlı işliyorum. Özellikle sorguların şişmesini engellemek için kritik sütunlarda **indekslemeler** yaptım. Bu sayede hem anlık kimlik doğrulamada hem de geçmişe dönük raporlamada sistemin performansını optimize etmiş oldum.

2.7 Authentication Mekanizmalari

Kimlik doğrulama tarafında sistemi iki ana kanal üzerinden kurguladım. Hem standart kullanıcılar hem de ekranı olmayan IoT cihazları için esnek bir yapı oluşturdum:

- **PAP (Şifre Tabanlı Doğrulama):** Klasik kullanıcı adı ve şifre ikilisiyle çalışıyor. Güvenlik için şifreleri asla açık metin olarak tutmadım; hepsini **bcrypt** ile hash'leyerek veritabanına kaydettim.
  Yazdığım `verify_password()` fonksiyonu hem bu modern hash'leri hem de test aşamasındaki düz metinleri tanıyabiliyor, bu da geliştirme sürecinde işimi çok kolaylaştırdı. Ayrıca sistemi brute-force saldırılarına karşı korumak için **Redis** tabanlı bir kalkan kurdum: Üst üste 5 kez yanlış şifre girilirse hesap 10 dakika boyunca otomatik kilitleniyor.
- **MAB (Cihaz Tabanlı Doğrulama):** Yazıcı, IP telefon veya kamera gibi kullanıcı girişi yapamayacak cihazlar için MAC adresi üzerinden bir çözüm

ürettim. Sistem, gelen isteği `is_mab_request()` fonksiyonuyla analiz ederek kullanıcı adının bir MAC adresi olup olmadığını anlıyor. Farklı cihazlardan gelen değişik MAC formatlarını (noktalı, tireli veya bitişik) `normalize_mac()` fonksiyonuyla tek bir standart formata (AA:BB:CC...) dönüştürdüm. Eğer cihaz veritabanında kayıtlıysa direkt onay alıyor; tanınmıyorsa güvenlik amacıyla kısıtlı bir "Guest" VLAN'ına yönlendiriliyor.

2.8 Authorization

Doğrulama aşaması geçildikten sonra işin en kritik kısmı olan **dinamik yetkilendirme** süreci devreye giriyor. Burada sistemi sabit kurallar üzerine değil, tamamen esnek bir **VLAN atama** mantığıyla kurguladım.
Sistemde şu an için dört ana grup tanımlı: **Admin** (VLAN 10), **Employee** (VLAN 20), **Guest** (VLAN 30) ve **IoT** (VLAN 40). Bir kullanıcı sisteme girdiğinde, hangi gruptaysa o gruba ait `Tunnel-Type`, `Tunnel-Medium-Type`, `Tunnel-Private-Group-Id` ve `Filter-Id` gibi RADIUS niteliklerini doğrudan `radgroupreply` tablosundan çekiyoruz.
Bu yapının en büyük avantajı bana **hareket alanı** sağlaması. Yeni bir departman açıldığında veya yeni bir erişim politikası gerektiğinde koda dokunmama gerek kalmıyor; sadece veritabanına yeni bir grup kaydı ekleyerek tüm ağı dinamik olarak yönetebiliyorum. Bu da sistemi hem ölçeklenebilir kılıyor hem de yönetimini ciddi anlamda kolaylaştırıyor.

2.9 Accounting

Ağ trafiğinin takibi ve raporlanması için kurguladığım **Accounting** sürecini üç temel aşamada yönetiyorum: **Start** (oturumun başladığı an), **Interim-Update** (bağlantı süresince gelen periyodik güncellemeler) ve **Stop** (oturumun kapandığı an).
Gelen tüm bu trafik verilerini kalıcı olarak PostgreSQL üzerindeki `radacct` tablosuna işliyorum. Ancak sadece veritabanıyla yetinmeyip performansı artırmak adına araya bir **Redis** katmanı ekledim. Aktif olan her oturumu, 24 saatlik bir ömürle (TTL) Redis üzerinde önbelleğe alıyorum.
Bu hibrit yapının bana sağladığı en büyük avantaj hız. Eğer "Şu an ağda kimler aktif?" diye bir sorgu atmak istersem, FastAPI üzerindeki `/sessions/active` endpoint'i ağır SQL sorgularıyla uğraşmak yerine doğrudan Redis'ten anlık yanıt dönebiliyor. Bir kullanıcı ağdan ayrılıp "Stop" paketi geldiğinde ise kayıt Redis'ten otomatik olarak temizleniyor ve döngü tamamlanmış oluyor.

3. Guvenlik Degerlendirmesi

Projenin güvenlik tarafında, sistemi hem dış saldırılara hem de iç zafiyetlere karşı korumak için birkaç farklı katmanda önlem aldım:

- **Şifre Güvenliği (Hashing):** Veritabanında hiçbir şifreyi açık metin (cleartext) olarak tutmuyorum. Tüm kullanıcı şifrelerini **bcrypt** algoritmasıyla hash'leyerek sakladım. Her şifre için rastgele bir **salt** değeri kullanıldığı için, iki kullanıcı aynı şifreyi seçse bile veritabanındaki hash'leri tamamen farklı görünüyor. Bu, "rainbow table" gibi saldırılara karşı ciddi bir koruma sağlıyor.

- **Siber Güvenlik (Brute-Force) Koruması:** Giriş denemelerini kontrol altında tutmak için **Redis** tabanlı bir **rate-limiting** mekanizması kurdum. Eğer bir hesap üzerinden üst üste 5 kez hatalı deneme yapılırsa, sistem o hesabı 10 dakika boyunca otomatik olarak kilitliyor. Giriş başarılı olduğunda ise bu sayaç sıfırlanıyor.

- **Hassas Veri Yönetimi:** Veritabanı şifreleri veya RADIUS gizli anahtarları (secret) gibi kritik bilgileri asla kodun içine gömmedim.
  Bunları `.env` dosyasında tutup `.gitignore` ile korumaya aldım.

  Başkalarının projeyi kurabilmesi için de hangi değişkenlerin gerektiğini anlatan bir `.env.example` dosyası hazırladım.

- **Ağ İzolasyonu ve Port Yönetimi:** Tüm servisleri Docker üzerinde oluşturduğum özel bir **bridge network** (`172.20.0.0/16`) içine hapsettim.

  Dış dünyaya sadece gerçekten ihtiyaç duyulan portları (RADIUS için 1812/1813, API için 8000 vb.) açtım. Böylece servisler kendi aralarında güvenli bir şekilde konuşurken, dışarıdan gelebilecek gereksiz erişim risklerini minimize ettim.

3.1 Potansiyel Riskler ve İyileştirme Önerileri

Projenin şu anki haliyle sunduğu çözümlerin yanında, gerçek dünya senaryolarında (production ortamında) geliştirilmesi gereken bazı kritik noktaları da raporun **"Risk Analizi ve İyileştirme Önerileri"** kısmında şu şekilde belirttim:

- **RADIUS Secret Güvenliği:** Şu an test aşamasında kullandığım `testing123` gibi basit bir anahtar, gerçek bir ağda ciddi bir zafiyettir. Canlı ortama geçişte mutlaka tahmin edilmesi zor, uzun ve karmaşık bir **shared secret** kullanılmalıdır.

- **İletişim Kanalı Güvenliği (TLS/HTTPS):** Mevcut yapıda API ile FreeRADIUS arasındaki trafik **HTTP** üzerinden akıyor. Veri güvenliğini bir üst seviyeye taşımak için bu trafiğin **TLS/HTTPS** katmanıyla şifrelenmesi, aradaki paketlerin izlenmesini (sniffing) engellemek adına şarttır.

- **Servis Erişim Kontrolleri:** PostgreSQL ve Redis gibi servisler şu an geliştirme odaklı yapılandırılmış durumda. Üretim ortamında bu servisler için sıkı şifre politikaları ve sadece belirli IP'lerin erişimine izin veren kısıtlamalar (IP whitelisting) devreye alınmalıdır.
- **İzleme ve Alarm Sistemleri (Monitoring):** Sistemin sağlığını anlık takip edebilmek ve şüpheli erişim denemelerini (anormal trafik artışları gibi) yakalayabilmek için **Prometheus** ve **Grafana** gibi monitoring araçlarının entegre edilmesi, projenin sürdürülebilirliği için çok önemli bir adım olacaktır.

4. Gerçek Dünya Uygulamaları

Geliştirdiğim bu NAC prototipi, sadece teorik bir çalışma değil, güncel ağ güvenlik ihtiyaçlarına doğrudan hitap eden bir altyapıya sahip. Sistemin kullanım potansiyelini şu üç ana başlıkta değerlendirebiliriz:

4.1 Kurumsal Ağ Güvenliği

- **4.1 Kurumsal Ağlarda Segmentasyon:** Büyük ölçekli şirketlerde en büyük risk, ağın "düz" (flat) olmasıdır. Bu sistem sayesinde personeli departmanına göre otomatik olarak farklı VLAN'lara ayırabiliyoruz. Örneğin; IT ekibi VLAN 10 üzerinden sunuculara tam erişim sağlarken, muhasebe birimi VLAN 20 ile sadece kendi yazılımlarına ulaşabiliyor. Misafirler ise VLAN 30'da kısıtlanarak sadece internete çıkabiliyor. Bu **segmentasyon** mantığı, olası bir siber saldırıda saldırganın ağ içinde yatayda ilerlemesini (**lateral movement**) teknik olarak imkansız hale getiriyor.

- **4.2 Eğitim Kurumları ve Kampüs Güvenliği:** Üniversite gibi çok farklı kullanıcı profillerinin (öğrenci, akademisyen, idari personel) bir arada bulunduğu yerlerde dinamik yetkilendirme hayat kurtarıcıdır. Sistem, üniversitenin mevcut kullanıcı veri tabanıyla (LDAP/AD) eşleşerek otomatik grup ataması yapabilir. Ayrıca kampüs içindeki akıllı tahta, projeksiyon veya yazıcı gibi kullanıcı girişi yapılamayan **IoT cihazlarını**, MAB (MAC tabanlı doğrulama) ile tanıyıp izole bir VLAN'da tutarak ana ağı bu cihazların zafiyetlerinden korumuş oluruz.

4.2 IoT ve Akıllı Bina Sistemleri: Günümüzde binalarda onlarca sensör, kamera ve kontrol sistemi ağa bağlı durumda. Bu cihazların çoğu gelişmiş güvenlik protokollerini (802.1X gibi) desteklemiyor. Geliştirdiğim MAB mekanizması tam da burada devreye giriyor. Bir güvenlik kamerasının ele geçirilmesi durumunda, saldırganın bina otomasyon sistemine (HVAC, aydınlatma vb.) sızması, cihazların farklı VLAN'lara hapsedilmesiyle engelleniyor.
Ayrıca Accounting kayıtları sayesinde hangi cihazın ne kadar trafik tükettiğini anlık izlemek de mümkün.

5. Sonuç ve Öğrenilenler

Bu proje süreci, Freeradius'u sıfırdan öğrenip aynı zamanda **RADIUS (RFC 2865/2866)** protokolünün sadece kağıt üzerindeki çalışma mantığını değil, **AAA mimarisinin** gerçek dünya senaryolarında nasıl hayata geçirildiğini kavramamı sağladı. Özellikle **FreeRADIUS** yapılandırması sırasında `rlm_rest` entegrasyonu ve "virtual server" yapısı üzerine çalışmak, ağ servislerinin ne kadar esnek yönetilebileceği konusunda bana ciddi bir derinlik kattı. API tarafında bilgim olduğu için bu yapıyı anlamam zor olmadı.

Teknik altyapıyı kurarken **Docker Compose** ile çoklu servis yönetimi, konteynerlar arası bağımlılıkların ayarlanması ve `healthcheck` mekanizmalarının kurgulanması konularında doğrudan pratik deneyim kazandım. Docker'a zaten önceden hakimdim fakat pratikleştirmemi sağladı

Özellikle FreeRADIUS'u Docker üzerinde ayağa kaldırırken yaşadığım **port çakışmaları**, **EAP modül hataları** ve **rlm_rest tarafındaki body encoding** sorunları, aslında teoride basit görünen entegrasyonların sahada ne tür zorluklar çıkarabileceğini anlamamda çok öğretici oldu. API'lerin çalışma mantığına başka bir açıdan bakma fırsatı buldum.

Projenin benim için en "öğretici" ve bir o kadar da zorlayıcı kısmı, **rlm_rest modülünün** HTTP yanıtlarını yorumlama biçimini çözmekti. API tarafında her şey yolunda gitmesine ve `200 OK` dönmeme rağmen FreeRADIUS'un isteği reddetmesi (reject) API'lerin çalışma mantığında derinlemesine inmemi sağladı.

Yaptığım debug çalışmaları sonucunda, modülün yanıt gövdesindeki verileri parse ederek durumu `updated` olarak işaretlediğini ve bunun da akışı bozduğunu fark ettim. **Başarılı doğrulamalarda boş bir body** `{}` **dönerek** bu sorunu çözmem, protokol düzeyinde hata ayıklama ve teknik dokümantasyonun "satır aralarını" okuma becerimi geliştirdi.

Son olarak, **Redis** kullanarak hem brute-force saldırılarına karşı `rate-limiting` hem de aktif oturumlar için `cache` mekanizması kurmak, bellek içi (in-memory) veri yapılarının performans üzerindeki etkisini görmemi sağladı. Redis mimarisine hakimdim fakat pratik yapmak da oldukça geliştiriyor.

Ayrıca **bcrypt** ile şifre güvenliğini sağlarken sistemin geriye dönük uyumluluğunu da bozmadan nasıl hash'leme yapılabileceğini, bcryp'in özellikle Brute Force saldırısını engellediğini, rate limiting koyup saldırıları önleyebileceğini gördüm. bu projeyle uygulamalı olarak öğrenmiş oldum.

6. Kaynaklar

- RFC 2865 — Remote Authentication Dial In User Service (RADIUS), C. Rigney et al.,2000
- RFC 2866 — RADIUS Accounting, C. Rigney, 2000
- FreeRADIUS Documentation — https://freeradius.org/documentation/
- FreeRADIUS rlm_rest Module —https://networkradius.com/doc/current/raddb/mods-available/rest.html
- FastAPI Documentation — https://fastapi.tiangolo.com/
- Docker Compose Documentation — https://docs.docker.com/compose/
- PostgreSQL 18 Documentation — https://www.postgresql.org/docs/
- Redis Documentation — https://redis.io/docs/
- bcrypt — https://pypi.org/project/bcrypt/
- asyncpg — https://magicstack.github.io/asyncpg/
- IEEE 802.1X — Port-Based Network Access Control